职航智管

AI 就业规划与求职事务智能管理平台

技术说明书

版本：v1.0 | 文档日期：2026-08-07 | 运行形态：Docker 全量部署

文档定位

本文面向评审、开发、测试和部署人员，说明当前项目的真实技术实现、边界和可验证能力。文档不把公开岗位检索说成稳定商业招聘 API，也不把模型建议说成自动办事结果。



图 1 学生工作台：目标岗位、投递漏斗、任务和提醒统一呈现

目录

目录 2

1. 项目概述 4

1.1 建设目标 4

1.2 用户角色和边界 4

2. 总体架构 5

2.1 运行时服务清单 5

2.2 一次请求的生命周期 6

3. 前端技术实现 7

3.1 React、TypeScript 与 Vite 7

3.2 页面信息架构 7

3.3 流式对话与前端状态 8

3.4 面试语音和摄像头 8

4. 后端与数据层 10

4.1 FastAPI 与 Pydantic 10

4.2 SQLAlchemy、Alembic 与 PostgreSQL 10

4.3 pgvector 检索 11

5. Agent 子系统详解 12

5.1 Agent 角色 12

5.2 工具目录和最小权限 13

5.3 会话、事件与动作状态 13

5.4 Provider 故障转移和用量记录 13

6. AI 和本地模型服务 15

6.1 在线大模型接口 15

6.2 BGE-M3 向量模型 15

6.3 PaddleOCR 15

6.4 FunASR 与 MeloTTS 15

6.5 YOLO 情绪与表达稳定度 15

7. 核心业务状态机 17

7.1 就业画像 17

7.2 求职计划和任务 17

7.3 投递状态 17

7.4 模拟面试 17

7.5 入职反馈 17

8. 安全、隐私和审计 18

9. 部署与运维 19

9.1 Docker 全量运行 19

9.2 配置分层 19

9.3 端口和健康检查 19

10. 测试与质量保障 21

10.1 测试层次 21

10.2 关键链路验收 21

11. 监控、成本和扩展 22

12. 项目目录与代码入口 23

12.1 重要接口示例 23

13. 真实能力与限制说明 24

14. 结论 24

附录 A：截图索引 25

1. 项目概述

职航智管面向高校应届毕业生和就业辅导员，围绕“就业画像—目标规划—岗位发现—简历与面试—投递跟踪—入职回访—就业质量改进”建立一条持续运行的就业事件链。系统将 AI 放在业务流程中，而不是只提供一个脱离上下文的聊天窗口。

1.1 建设目标

- 把学生分散的简历、岗位、投递、面试、学习和入职反馈沉淀为可追踪记录。
- 用证据支持画像、岗位匹配、面试评分和复盘建议，降低模型幻觉对业务的影响。
- 让辅导员依据可解释的风险事实进行分层帮扶，而不是只看最终就业结果。
- 让 Agent 能够规划和调用工具，同时由后端掌握角色、资源归属和高风险动作授权。

1.2 用户角色和边界

角色	主要页面	可以做什么	明确限制
学生	学生工作台、就业助手、画像、计划、岗位、面试、职场训练、入职跟踪	管理本人求职事实，接受 AI 建议，确认计划和动作	不能访问其他学生数据，不能绕过高风险确认
辅导员	班级概览、学生进展、帮扶任务、就业质量、Agent 统计、审计日志	查看授权范围内学生事实，发起帮扶，审查就业质量	只能访问所属班级或授权学生范围
企业联系人	一次性企业评价页	提交入职后表现和岗位反馈	链接一次性、限时、提交后失效
系统管理员	部署与治理配置	配置服务、来源适配器和审计策略	不通过前端替代业务审批

2. 总体架构

系统采用前后端分离和服务化部署。浏览器通过 Nginx 访问 React 前端和 `/api` 反向代理；FastAPI 负责鉴权、业务编排、Agent 网关和任务状态；PostgreSQL 保存业务事实与审计数据，pgvector 保存向量；Redis 和 Worker 承担异步任务；BGE-M3、PaddleOCR、FunASR、MeloTTS 和情绪分析作为独立服务运行。

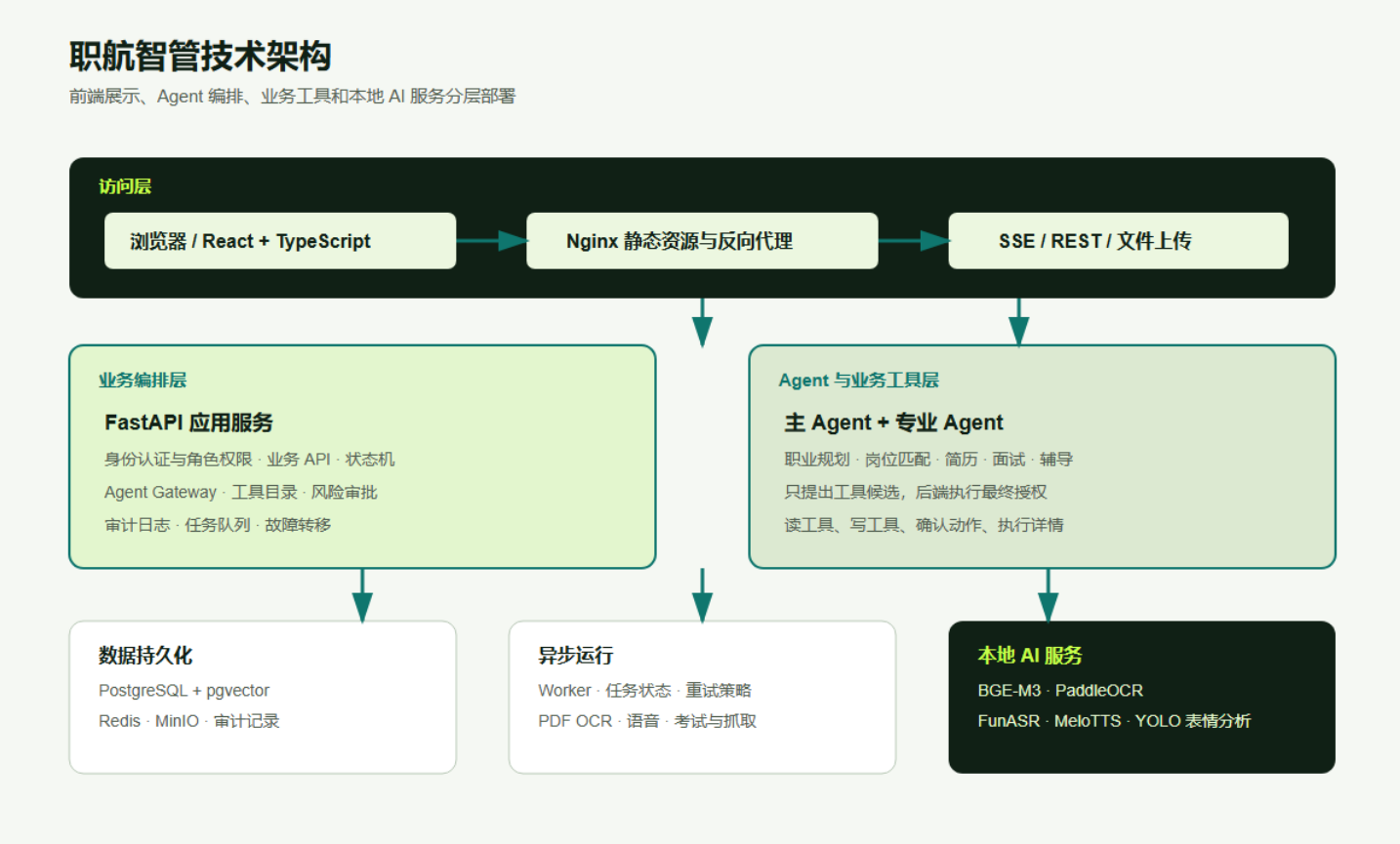


图 2 分层技术架构与独立 AI 服务边界

2.1 运行时服务清单

服务	技术/镜像	职责	健康检查与故障边界
frontend	React + Vite + Nginx	静态资源、路由、SSE 客户端、响应式界面	只负责展示和请求，不执行权限判断
backend	FastAPI + SQLAlchemy	认证、业务 API、Agent 编排、工具授权、审计	健康检查依赖数据库和核心配置
worker	Python Worker + Redis 队列	OCR、ASR、TTS、向量化和抓取任务	任务状态 queued/running/succeeded/failed

服务	技术/镜像	职责	健康检查与故障边界
db	PostgreSQL + pgvector	业务表、会话、消息、向量、审计记录	数据卷持久化，Alembic 管理迁移
embedding	BGE-M3 独立服务	文本向量化与语义检索	查询量不大时优先 CPU 稳定运行
ocr	PaddleOCR 独立服务	扫描 PDF 和图片文字识别	所有 PDF 强制进入 OCR 解析链路
speech	FunASR + MeloTTS	语音识别、数字人问题朗读	语音服务可使用 GPU，TTS 默认 CPU
emotion	YOLO 人脸检测 + 本地表情模型	面试表达稳定度辅助指标	浏览器主动授权，服务只保存统计结果

2.2 一次请求的生命周期

1. 浏览器携带 JWT 和请求幂等键调用 `/api`。
2. FastAPI 解析用户、角色、资源上下文和请求参数。
3. 读操作直接查询；耗时操作创建 AI 任务并由 Worker 异步处理。
4. Agent 请求通过会话网关进入主 Agent，产生状态事件和工具候选。
5. 授权网关重新验证角色、资源归属、参数契约和风险等级。
6. 工具执行结果脱敏后写入执行详情、审计日志和业务表。
7. 前端通过 SSE 或查询轮询恢复状态、消息和最终结论。

3. 前端技术实现

3.1 React、TypeScript 与 Vite

前端入口位于 `frontend/src`，以 React 函数组件实现页面，以 TypeScript 为业务数据和接口契约提供静态类型约束，以 Vite 负责开发服务器和生产构建。页面采用路由懒加载，避免所有功能在首屏同时加载。

技术	实现细节	解决的问题
React	页面拆分为学生、辅导员、公共评价、Agent 对话和面试会话组件	降低复杂业务页面的耦合度
TypeScript	接口响应、Agent 事件、面试评分、任务状态均有类型定义	减少字段拼写和状态分支错误
Vite	生产构建输出带哈希的 JS/CSS 资源，Nginx 提供静态文件	提高加载性能，避免旧资源覆盖新资源
React Router	按角色保护 `/student/*`、`/counselor/*` 和公开评价路由	实现页面级权限和独立会话页
TanStack Query	缓存查询、失效刷新、Mutation 状态和重试	统一处理异步业务数据
Lucide	按钮、导航、状态、风险和业务动作统一使用图标	提高识别效率并保持视觉一致

3.2 页面信息架构

业务阶段	路由	核心组件/数据
总览与对话	`/student`、`/student/assistant`	工作台指标、持久会话、执行过程
规划成长	`/student/profile`、`/student/plan`、`/student/learning`	画像证据、四周计划、能力雷达和学习证据
求职行动	`/student/jobs`、`/student/intelligence`、`/student/resume`	五维匹配、公开岗位来源、简历解析
面试训练	`/student/interview`、`/student/interview/:sessionId`	配置页与独立对话页、流式评分、语音和摄像头
入职后成长	`/student/workplace`、	场景状态机、简报、交付物和验收

业务阶段	路由	核心组件/数据
	`/student/workplace/session/:sessionId`	
闭环反馈	`/student/applications`、`/student/employment`、`/student/review`	投递阶段、7/30/90/180 天回访、周复盘
辅导员治理	`/counselor/*`	班级风险、帮扶、就业质量、Agent 统计和审计

3.3 流式对话与前端状态

就业助手使用“乐观用户消息 + 服务端运行状态 + 持久消息恢复”的组合方式。发送后先显示用户消息，随后创建 Agent Run；前端订阅事件 URL，按顺序处理 run.started、agent.started、tool.started、tool.completed、assistant.delta、run.completed 或 run.failed。服务端持久化成功后，TanStack Query 失效并重新读取会话，避免刷新丢消息或同一回答显示两遍。

3.4 面试语音和摄像头

- 录音：浏览器 MediaRecorder 采集 WebM，上传到 `/interviews/{id}/recordings`，后端创建异步识别任务。
- 朗读：问题提交到 speech 任务，Worker 调用 MeloTTS，前端轮询 artifact 并通过 audio 播放。
- 摄像头：用户主动授权后每 2.5 秒采样一次，图片缩放后上传，服务返回人脸存在率、主导表情和表达稳定度。
- 失败处理：麦克风、摄像头、ASR 或 TTS 不可用时，文字回答仍然可用，页面显示明确错误。



图 3 数字人面试会话：中文题目、数字人、进度、评分和摄像头能力

4. 后端与数据层

4.1 FastAPI 与 Pydantic

后端模块位于 `backend/app`，以领域目录组织 students、jobs、resumes、interviews、agents、chat、growth、workplace、employment 和 audit。FastAPI 路由负责 HTTP 边界，Pydantic 模型负责请求体约束和响应序列化，领域 service 负责事务和业务规则。

错误统一转换为带 code、message 和 request_id 的 API 错误。前端可以根据 `chat_run_active`、`request_timeout`、`tool_not_allowed`、`resource_forbidden` 等错误码显示可操作提示，而不是只显示“请求失败”。

4.2 SQLAlchemy、Alembic 与 PostgreSQL

SQLAlchemy ORM 映射业务实体，数据库会话通过 FastAPI 依赖注入获得。写入服务支持由 API 请求或 Agent 编排器控制提交：普通 API 在请求结束提交，Agent 工具只 flush，所有动作结束后统一 commit，失败统一 rollback，避免出现半成品数据。Alembic 迁移文件按版本管理从空库到完整系统的结构变化。

数据域	关键实体	主要作用
身份与权限	User、Role、ClassScope、AuditLog	登录、角色、班级范围、操作审计
学生求职	StudentProfile、ResumeFile、CareerPlanRecord、PlanTask	画像、简历、计划和任务事实
岗位与投递	Job、JobSource、CrawlRun、Application	岗位来源、时效、匹配和状态机
Agent 对话	ChatSession、ChatMessage、AgentRun、ToolCall、ActionProposal	持久会话、运行过程和待确认动作
模拟面试	InterviewSession、InterviewTurn、MediaArtifact	题目计划、回答、评分、音频和情绪汇总
入职闭环	EmploymentRecord、Followup、	回访节点、双向反馈和就业质量

数据域	关键实体	主要作用
	EmployerInvitation、EmployerReview	

4.3 pgvector 检索

BGE-M3 为简历片段、岗位描述、课程材料和业务知识生成向量，向量写入 PostgreSQL 的 pgvector 列。查询时先按用户权限和业务上下文过滤，再进行相似度检索，最后把来源 ID 和引用片段返回给 Agent。向量只负责召回，不替代业务权限校验。

```
查询流程：权限过滤 → 文本分段 → BGE-M3 embedding → pgvector 相似度 → 来源重排 → Agent 引用
```

5. Agent 子系统详解

Agent 子系统由会话网关、主 Agent、专业 Agent、工具目录、动作确认、执行记录和模型 Provider 组成。核心原则是“模型只做规划，后端做授权和执行”。

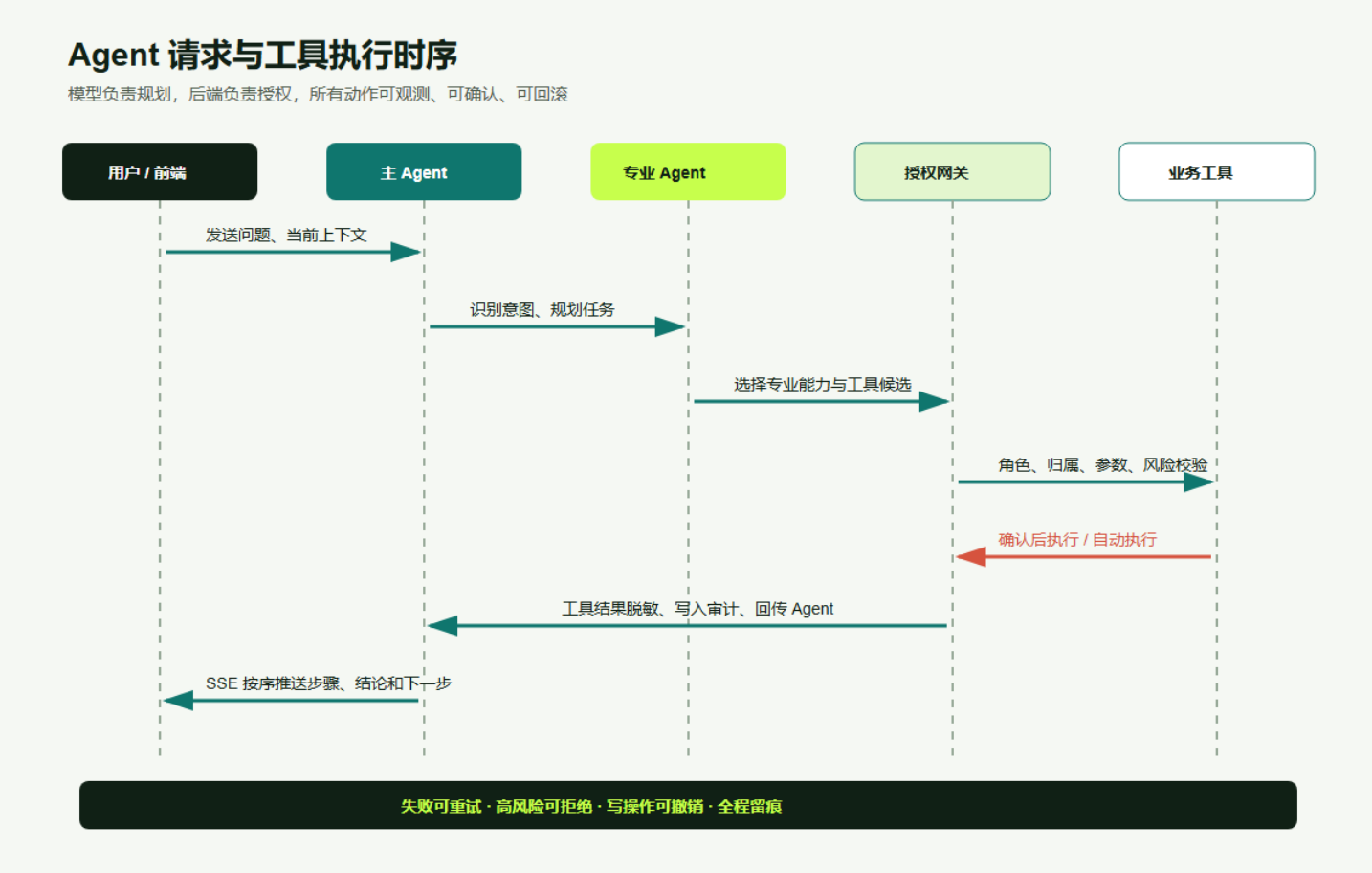


图 4 Agent 工作流：候选工具必须经过授权网关

5.1 Agent 角色

Agent	输入上下文	输出
主 Agent	用户问题、当前会话、业务上下文	意图、执行计划、专业 Agent 路由和综合回答
职业规划 Agent	画像、目标岗位、计划任务	优先行动、计划调整建议和依据
岗位 Agent	画像、岗位详情、岗位来源	匹配度、差距、投递建议和岗位解释
简历 Agent	简历版本、岗位要求、证据片段	修改建议、证据缺口和风险提示
面试 Agent	题目计划、回答、历史评分	动态追问、维度评分和改进反馈
辅导员 Agent	授权学生范围、风险事实、帮扶记录	干预建议、任务草案和班级汇总

Agent	输入上下文	输出
职场训练 Agent	场景简报、交付材料、验收标准	交付物草案、检查清单和复盘建议

5.2 工具目录和最小权限

工具是后端注册的结构化能力，每个工具包含名称、版本、可用角色、适用 Agent、参数 JSON Schema、风险等级、资源范围和执行函数。模型不能通过自然语言创建新工具，也不能把未注册工具写进调用请求。

工具类别	示例	风险策略
只读工具	profile.get、plan.get、tasks.list、applications.list	通过角色和资源范围后直接执行
分析工具	jobs.match、knowledge.search、growth.overview	记录引用来源和模型用量
普通写工具	plan.confirm、task.complete、application.stage	确认模式等待用户；自动模式可执行
高风险工具	delete、batch_update、external_submit	无论模式都必须二次确认或保持禁用

5.3 会话、事件与动作状态

状态	含义	前端表现
queued	运行已创建，等待 Worker 或模型	显示等待队列
running	主 Agent 或工具正在处理	逐步显示 Agent/工具事件
pending_confirmation	生成了需要用户确认的动作	显示确认、拒绝、撤销按钮
succeeded	回答和工具结果已持久化	显示结论、来源和耗时
failed	模型、工具或契约校验失败	显示错误码、原因和重试入口

5.4 Provider 故障转移和用量记录

模型调用通过 OpenAI 兼容客户端抽象，Provider、模型名、超时、重试次数和响应解析彼此分离。正常路径使用配置的在线模型；发生连接超时、401、429 或 5xx 时按策略切换备用 Provider；所有

Provider 都不可用时返回明确错误或本地规则结果，并标注降级状态。每次调用记录 provider、model、输入输出 token、耗时、状态和错误码，便于成本与质量分析。

重要边界

本项目允许 Agent 生成建议和动作提案，但不自动替学生向外部招聘平台投递，不绕过登录、验证码或访问限制。高风险写操作保持人工确认，属于产品安全设计而不是功能缺失。

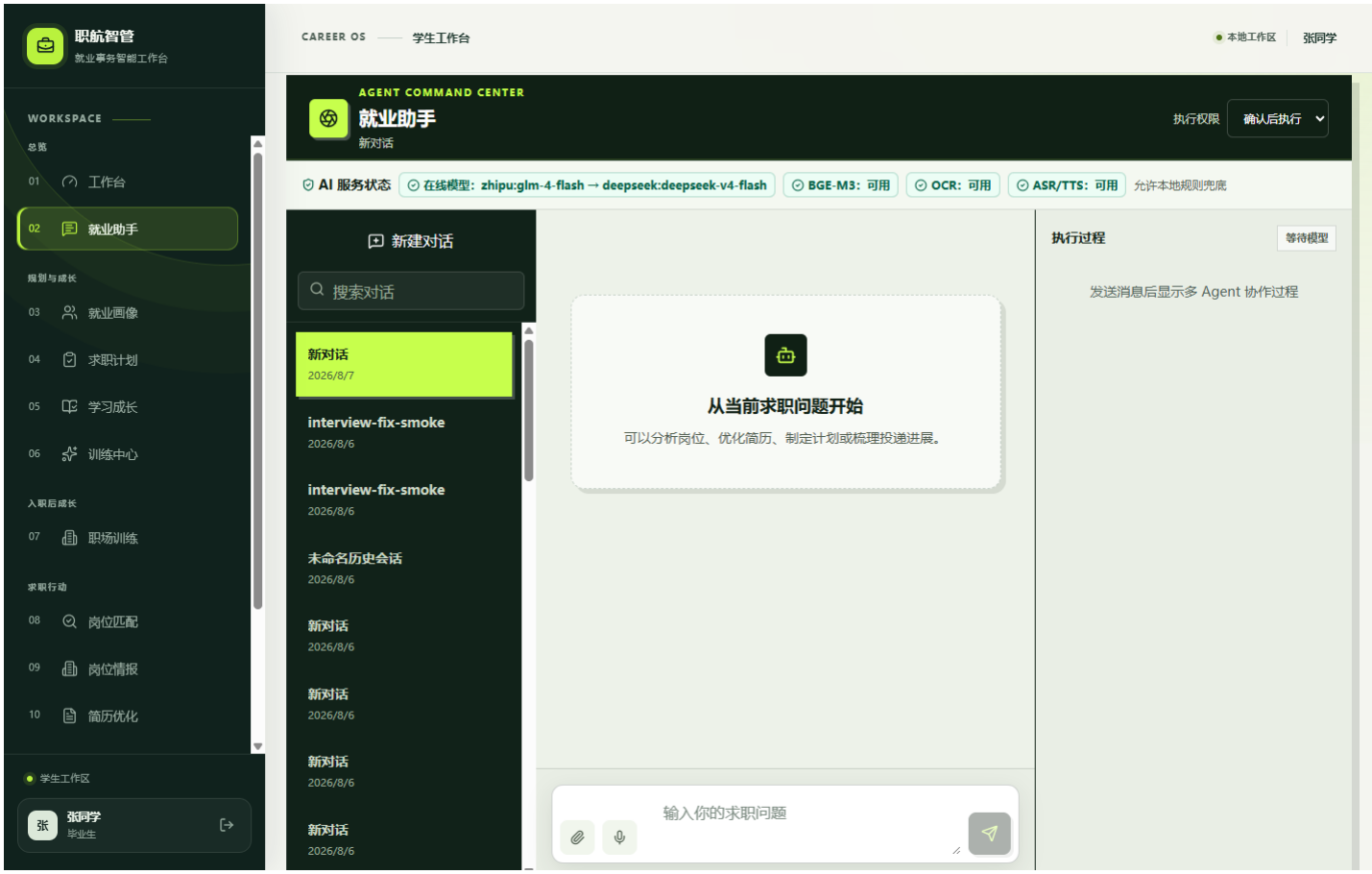


图 5 就业助手：持久会话、服务状态、执行权限和执行过程同屏可观测

6. AI 和本地模型服务

6.1 在线大模型接口

在线模型通过 OpenAI 兼容协议接入，业务代码不绑定某一家 SDK。配置项包括 base_url、API key、模型名、连接超时、读取超时和备用 Provider。API key 只从环境变量或密钥管理系统读取，不写入前端、不写入日志、不进入技术文档。

6.2 BGE-M3 向量模型

BGE-M3 作为独立 CPU 服务运行，负责中英文混合文本的向量化。文档、简历和岗位被分段后写入向量索引，检索结果带来源和引用片段，Agent 只能引用被检索到的内容。查询量不大时使用 CPU 是稳定性取舍；需要更高吞吐时再切换 GPU。

6.3 PaddleOCR

PDF 解析策略是所有 PDF 强制进入 OCR 链路。文本型 PDF 仍可先提取文字作为快速路径，扫描 PDF 则由 PaddleOCR 识别页面图像；结果进入统一的简历解析结构，记录页码、置信度和解析状态。大文件通过 Worker 异步执行，前端显示 queued/running/succeeded/failed。

6.4 FunASR 与 MeloTTS

FunASR 服务接收录音文件并返回转写文本，学生可以在提交前检查或修改转写内容。MeloTTS 用于数字人问题朗读，音频生成作为异步任务执行，前端在 artifact 可用后播放。两项能力均有降级路径：语音服务不可用时仍可文字回答和阅读题目。

6.5 YOLO 情绪与表达稳定度

摄像头分析不是心理诊断，也不输出“是否适合就业”的结论。浏览器仅在学生主动开启后采样，后端把每帧结果汇总为人脸存在率、主导表情、置信度和表达稳定度；最终报告将其作为可选维度，与内容评分分开计算。

服务	推荐运行位置	显存/资源建议	业务失败时的可用路径
BGE-M3	独立 CPU 容器	查询量不大时无需显卡	关键词检索或明确提示向量服务不可用

服务	推荐运行位置	显存/资源建议	业务失败时的可用路径
PaddleOCR	独立 CPU 容器	扫描 PDF 以异步为主	提示解析失败，允许手工录入
FunASR	独立 GPU 容器	RTX 3050 6GB 可运行轻量模型，控制并发	文字回答
MeloTTS	默认 CPU，可切 GPU	单条短音频优先 CPU	直接阅读文字题目
YOLO/表情模型	独立服务	按采样频率限制显存和上传量	不启用摄像头维度

7. 核心业务状态机

7.1 就业画像

```
draft → pending_confirmation → confirmed
失败状态：analysis_failed；确认后才写入正式 StudentProfile
```

7.2 求职计划和任务

```
plan: generated → pending_confirmation → active → archived
task: pending → in_progress → completed / blocked
```

7.3 投递状态

```
saved → applied → assessment → interview → offered
                    ↘ rejected / withdrawn
```

7.4 模拟面试

```
active → completed
每轮：question → answer → evaluation → next_question
允许 follow_up，但受 max_follow_ups 和 max_turns 约束
```

7.5 入职反馈

```
offered application → EmploymentRecord → day_7/day_30/day_90/day_180 follow-up
student feedback + employer review → moderation → quality analytics
```

8. 安全、隐私和审计

控制点	实现方式	验收重点
身份认证	JWT、密码哈希、登录审计、角色保护路由	错误密码不泄露用户是否存在
资源隔离	所有学生资源按 student_id 校验，辅导员按班级范围过滤	跨学生、跨班级访问返回拒绝
工具授权	工具白名单、参数契约、风险等级和资源范围	模型不能扩大工具权限
高风险动作	动作提案、确认/拒绝/撤销、二次确认	删除、批量修改和外部投递默认不自动执行
敏感数据	日志脱敏、简历正文不写入普通日志、企业评价匿名阈值	导出和展示前再次脱敏
一次性邀请	随机令牌、过期时间、提交后失效	重复提交被拒绝
数据保留	审计和业务数据分层保留，文件可迁移 MinIO/S3	定期清理策略可配置

安全设计的关键是“前后端双重校验”。前端按钮隐藏或禁用只能改善体验，不能作为安全边界；真正的角色、归属、参数和风险判断全部在后端完成。

9. 部署与运维

9.1 Docker 全量运行

正式演示推荐 Docker 全量模式，前端、后端、数据库、Redis、Worker、Embedding、OCR、Speech、Emotion、MinIO、Prometheus 和 OpenTelemetry 在同一网络中运行。这样可以避免“本地 8000 只连到部分服务，而 Docker 8080 是另一套系统”的不一致。

```
cd /d D:\table\ai_jiuye
docker compose up -d --build
docker compose ps
docker compose logs -f backend
```

9.2 配置分层

配置类别	典型变量	原则
数据库	DATABASE_URL、POSTGRES_PASSWORD	生产环境使用独立强密码和持久卷
在线模型	OPENAI_BASE_URL、OPENAI_API_KEY、MODEL_NAME	只在后端和 Worker 使用，禁止提交仓库
本地服务	EMBEDDING_URL、OCR_URL、SPEECH_URL、EMOTION_URL	容器内部使用服务名，宿主机只暴露必要端口
存储	STORAGE_BACKEND、MINIO_ENDPOINT、STORAGE_BUCKET	简历、音频和导出文件使用对象存储
观测	OTEL_EXPORTER_OTLP_ENDPOINT、PROMETHEUS	记录耗时、失败率和模型用量，不记录敏感正文

9.3 端口和健康检查

入口	用途	状态判断
8080	用户访问前端	浏览器能打开登录页，静态资源 200
8000（容器内）	FastAPI API	backend healthy，`/api/health` 正常
8101	BGE-M3	embedding health ready
8102	PaddleOCR	ocr health ready
8103	FunASR/MeloTTS	speech health ready

入口	用途	状态判断
8104	情绪分析	emotion health ready
5432/6379	PostgreSQL/Redis	容器健康且数据卷存在

10. 测试与质量保障

10.1 测试层次

层次	覆盖内容	证据
单元测试	评分量表、题目中文兜底、情绪汇总、工具契约	pytest 领域测试
服务测试	权限、事务、状态机、跨学生和跨班级隔离	FastAPI 测试客户端 + 临时数据库
Agent 回归	工具白名单、执行详情、动作确认、失败重试	Agent 专项测试集
前端构建	TypeScript、Vite、静态资源和路由懒加载	`npm run build`
浏览器 E2E	登录、页面渲染、真实会话、移动端布局和控制台	Playwright/agent-browser 截图与 DOM 快照
部署验收	Docker 服务健康、入口页面、AI 服务状态	`docker compose ps` 和健康检查

10.2 关键链路验收

- 上传 PDF → OCR → 简历解析 → 画像草稿 → 学生确认 → Agent 引用。
- 录音 → FunASR → 学生确认转写 → 面试评分 → MeloTTS 朗读下一题。
- 岗位来源刷新 → 采集记录 → 新鲜度检查 → 岗位情报 → 五维匹配。
- 录用投递 → 入职记录 → 7/30/90/180 天回访 → 学生反馈 + 企业评价 → 审核分析。
- 模型超时/401/429/500 → 重试或 Provider 切换 → 记录真实失败原因。

当前交付边界

容器 healthy 只代表服务进程可用，不能替代全链路业务验收。生产上线前仍应在目标机器上执行空库迁移、大文件 OCR、噪声音频、并发、限流和模型故障演练。

11. 监控、成本和扩展

- Prometheus 采集 API 请求量、延迟、错误率、队列长度和服务健康状态。
- OpenTelemetry 关联 request_id、chat_run_id、tool_call_id 和异步 task_id。
- Agent 统计按 Provider、模型、Agent、工具记录调用次数、Token、耗时和失败率。
- 失败任务进入可重试状态；多次失败后进入死信队列并等待人工处理。
- 文件和音频从本地目录迁移 MinIO/S3 后，API 只保存 artifact 元数据和访问权限。
- 并发提升时，通过 Redis 限流、Worker 横向扩展、数据库连接池和向量索引优化控制压力。

12. 项目目录与代码入口

```
frontend/src/app/                  路由、会话、API 客户端
frontend/src/features/chat/          Agent 对话、执行面板、动作确认
frontend/src/features/student/       学生业务页面、面试和职场训练
frontend/src/features/counselor/     辅导员页面、质量和审计
backend/app/agents/                  Agent 契约、Prompt、Provider、工具授权
backend/app/tools/                   工具目录和工具实现
backend/app/interviews/              面试模型、服务、路由和评分
backend/app/growth/                  学习任务、题库和考试
backend/app/workplace/               入职后场景状态机和交付物
backend/app/employment/              入职记录、回访、企业邀请
backend/alembic/versions/             数据库迁移
services/                              BGE、OCR、Speech、Emotion 独立服务
docker-compose.yml                    全量运行拓扑
```

12.1 重要接口示例

```
POST /api/chat/sessions
POST /api/chat/sessions/{id}/messages
GET  /api/chat/runs/{run_id}/events
POST /api/interviews
POST /api/interviews/{id}/answers/stream
POST /api/intelligence/sources/refresh?force=true
POST /api/jobs/{id}/match
POST /api/employment/records/{id}/employer-invitations
```

所有写接口都应携带 Authorization，外部动作还需要确认令牌或动作 ID。接口响应中的 request_id、task_id 和 run_id 用于定位日志和执行详情。

13. 真实能力与限制说明

能力	当前实现	不能对外宣称的内容
在线模型	OpenAI 兼容 Provider、超时、重试、切换和用量记录	不能保证第三方免费额度或长期稳定
公开岗位	来源适配器、刷新、导入、来源和新鲜度记录	不是稳定商业招聘 API，不自动投递
OCR/语音	独立容器和异步任务接口	健康检查通过不等于所有复杂文件和噪声都已覆盖
情绪分析	人脸/表达稳定度辅助维度	不是心理诊断，不用于就业预测或自动淘汰
Agent 写操作	动作提案、确认、撤销和审计	高风险动作不会无确认自动执行

在比赛演示中，应明确区分“真实服务调用”“本地规则降级”和“预置演示数据”。任何服务不可用时，页面应显示状态和错误码，不用默认文案伪装为真实分析。

14. 结论

职航智管的技术重点不是堆叠模型，而是把模型、业务数据、工具授权和人机协作组织成可审计的闭环。React 负责清晰的双端体验，FastAPI 负责业务和 Agent 编排，PostgreSQL/pgvector 负责事实与检索，独立 AI 服务负责 OCR、向量、语音和视觉能力，Docker 和观测组件保证运行可复现。

系统最终交付的价值是：学生知道下一步做什么，辅导员知道为什么要干预，Agent 能解释自己做了什么，学校能够用入职后的真实反馈改进课程和就业服务。

附录 A：截图索引



图 6 岗位情报：公开来源、岗位新鲜度、热门岗位和活跃企业

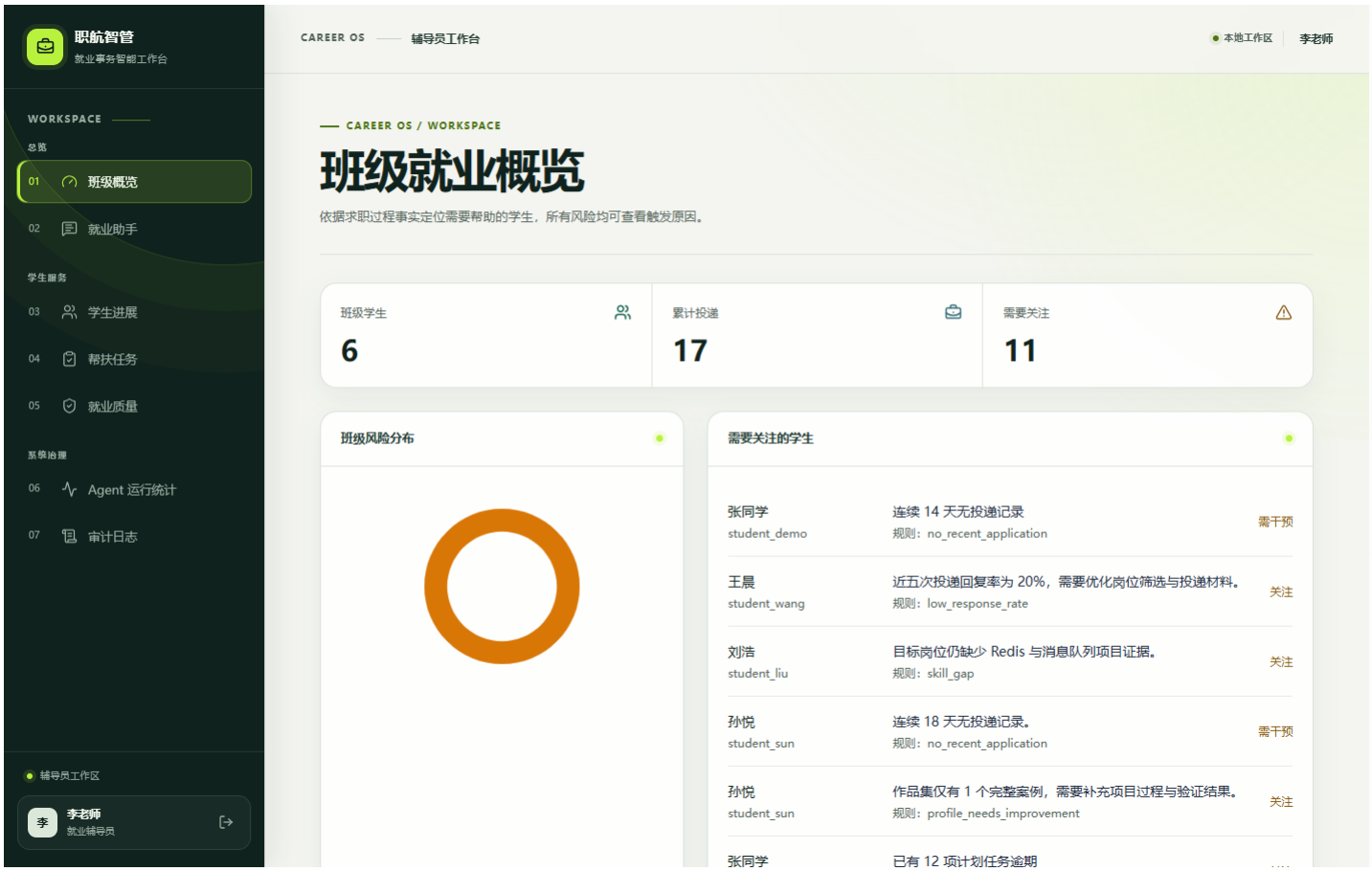


图 7 辅导员班级概览：风险分布和可解释的学生关注列表